

Step 1 – Prompt for AI Coding Assistant (Copilot)**Prompt:**

Create a simple React functional component called GuestList that consumes a REST API endpoint using fetch().

Use the endpoint <https://jsonplaceholder.typicode.com/users>.

The component should:

- Fetch the guest data when the component loads.
- Display the guest names in a list.
- Show "**Loading guests...**" while the data is being fetched.
- Show an error message if the API request fails.
- Use React hooks such as useState and useEffect.
- Keep the code simple and beginner-friendly.
- Do not use any external libraries.

Step 2 – Component Generated

```
import { useEffect, useState } from 'react';
```

```
function GuestList() {  
  
  const [guests, setGuests] = useState([]);  
  
  const [loading, setLoading] = useState(true);  
  
  const [error, setError] = useState('');  
  
  
  useEffect(() => {  
  
    async function fetchGuests() {  
  
      try {  
  
        const response = await fetch('https://jsonplaceholder.typicode.com/users');  
  
  
        if (!response.ok) {  
  
          throw new Error('Failed to load guests.');
```

```
}

const data = await response.json();

setGuests(data);
} catch (err) {
  setError(err.message || 'Something went wrong while loading guests.');
```

```
} finally {
  setLoading(false);
}
}
```

```
fetchGuests();
}, []);
```

```
if (loading) {
  return <p>Loading guests...</p>;
}
```

```
if (error) {
  return <p>{error}</p>;
}
```

```
return (
  <div>
    <h2>Guest List</h2>
    <ul>
      {guests.map((guest) => (
        <li key={guest.id}>{guest.name}</li>
```

```
    )})  
  </ul>  
</div>  
);  
}
```

```
export default GuestList;
```

Step 2 – Add the Component

I created a React component called GuestList.jsx and added the code generated by Copilot.

The component uses:

- useState to store guests, loading status, and errors.
- useEffect to call the REST API when the component loads.
- fetch() to get the guest data.
- Loading and error states to handle different API conditions.

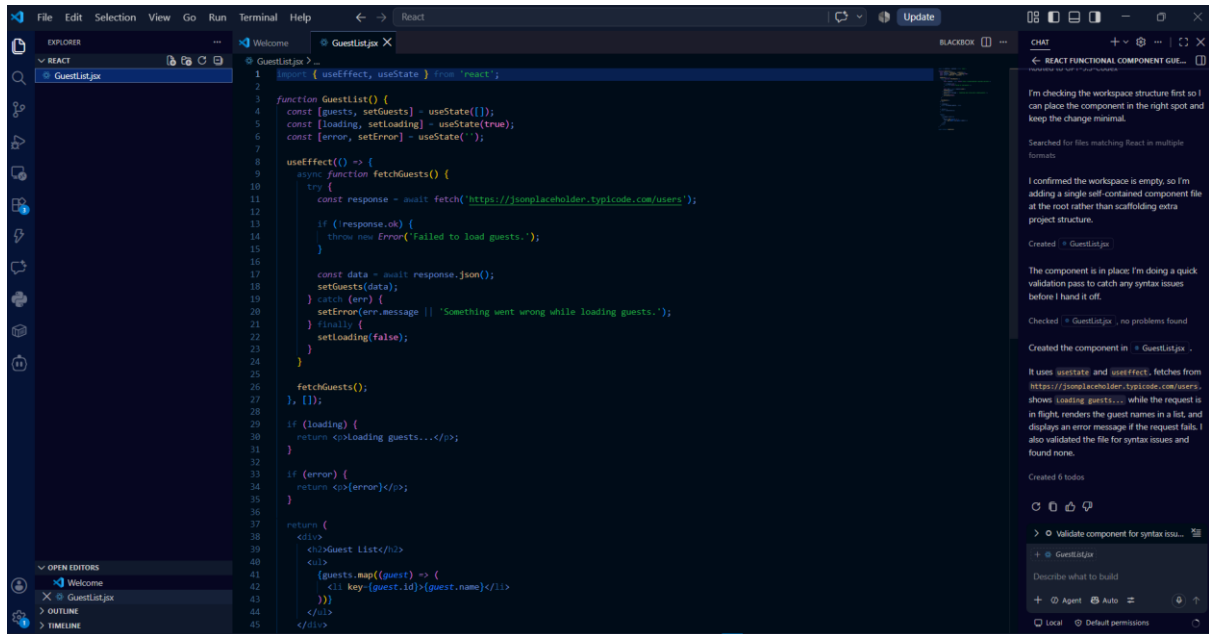
Step 3 – Display the Component

In App.jsx, import and use the component:

```
import GuestList from './GuestList';
```

```
function App() {  
  return (  
    <div>  
      <GuestList />  
    </div>  
  );  
}
```

export default App;



Step 4 – Test the REST API

I opened the application and verified that the component successfully fetched data from the REST endpoint.

The guest names were displayed under **Guest List**.

Expected result:

Guest List

Leanne Graham

Ervin Howell

Clementine Bauch

Patricia Lebsack

Chelsey Dietrich

...

Step 5 – Test Loading State

When the API request is being processed, the component displays:

Loading guests...

This is controlled by:

```
if (loading) {
  return <p>Loading guests...</p>;
}
```

Step 6 – Test Error State

To test the error handling, temporarily change the API URL to an invalid endpoint:

```
const response = await fetch(
  'https://jsonplaceholder.typicode.com/invalid'
);
```

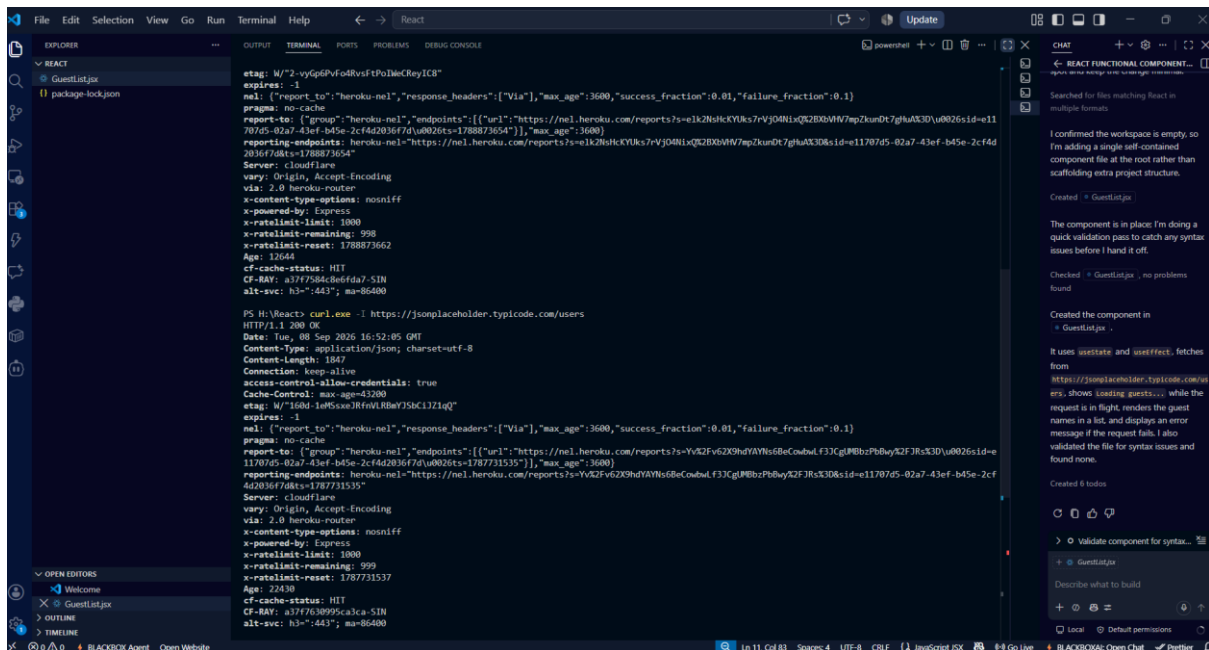
Run the application again.

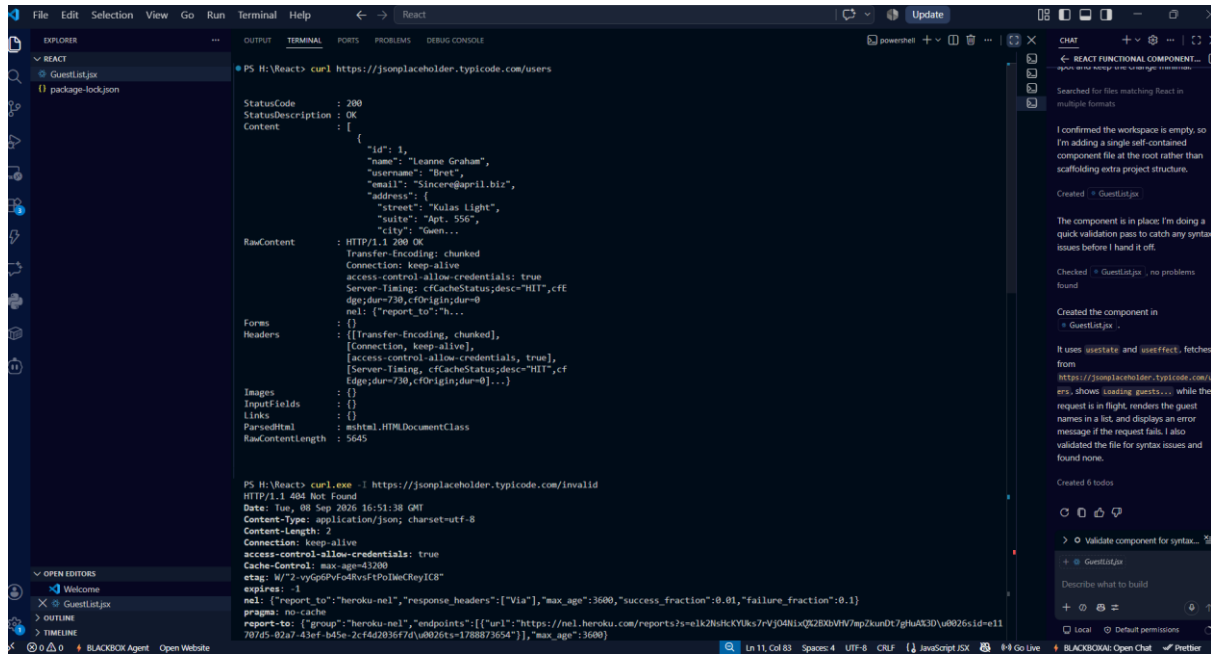
The component should display:

Unable to load guests.

After testing, change the URL back to:

<https://jsonplaceholder.typicode.com/users>





The screenshot shows a VS Code editor interface with a React project. The Explorer panel on the left shows the file structure with 'GuestList.jsx' selected. The main editor area displays the REST client for 'https://jsonplaceholder.typicode.com/users'. The first request is a GET request to '/users' with a status of 200. The response is a JSON array of user objects. The second request is a GET request to '/invalid' with a status of 404. The right sidebar shows a chat window with a message from 'React Functional Component...' and a 'Validate component for syntax...' button.

```
PS H:\React> curl https://jsonplaceholder.typicode.com/users

StatusCode : 200
StatusDescription : OK
Content : [{"id": 1, "name": "Leanne Graham", "username": "Bret", "email": "Sincere@april.biz", "address": {"street": "Kulas Light", "suite": "Apt. 556", "city": "Gwen..."}, "RawContent": "HTTP/1.1 200 OK\r\nTransfer-Encoding: chunked\r\nConnection: keep-alive\r\naccess-control-allow-credentials: true\r\nServer-Timing: cfCacheStatus=desc=HIT,cfEdge;dur=730,cfOrigin;dur=0\r\nnel: {\"report_to\": \"heroku-nel\", \"response_headers\": [\"Via\"], \"max_age\": 3600, \"success_fraction\": 0.01, \"failure_fraction\": 0.1}, \"pragma\": \"no-cache\", \"endpoints\": [{\"url\": \"https://nel.heroku.com/reports?selk2N8HcKYUks7VjJ04NiXQZB0XVW7mpZkxuD7gHuAX3Dv0026sId=e11707d5-02a7-43ef-b45e-2c442036f7dvd0026ts=1788873654\"}], \"max_age\": 3600}"}]

PS H:\React> curl.exe -i https://jsonplaceholder.typicode.com/invalid
HTTP/1.1 404 Not Found
Date: Tue, 08 Sep 2026 16:51:38 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 2
Connection: keep-alive
access-control-allow-credentials: true
Cache-Control: max-age=3200
etag: W/"2-vyq6Pv0d8vsFtPoMcReyIC8"
expires: -1
nel: {\"report_to\": \"heroku-nel\", \"response_headers\": [\"Via\"], \"max_age\": 3600, \"success_fraction\": 0.01, \"failure_fraction\": 0.1}, \"pragma\": \"no-cache\", \"endpoints\": [{\"url\": \"https://nel.heroku.com/reports?selk2N8HcKYUks7VjJ04NiXQZB0XVW7mpZkxuD7gHuAX3Dv0026sId=e11707d5-02a7-43ef-b45e-2c442036f7dvd0026ts=1788873654\"}], \"max_age\": 3600}
```